

棕熊身份识别流水线 — 技术文档

作者: Darian Ding 日期: 2026 年 5 月 4 日 目标读者: 团队成员、技术 mentor、未来维护者 对应代码分支: `feature/auto-annotation`

1. 项目背景

1.1 动机

Winter Quarter 的设计报告中, "跨视频识别同一头熊" (cross-video bear identification) 这项需求由于 ByteTrack 仅做帧间运动学跟踪、没有外观特征 Re-ID 网络, 被从项目范围中移除。直接结果是:

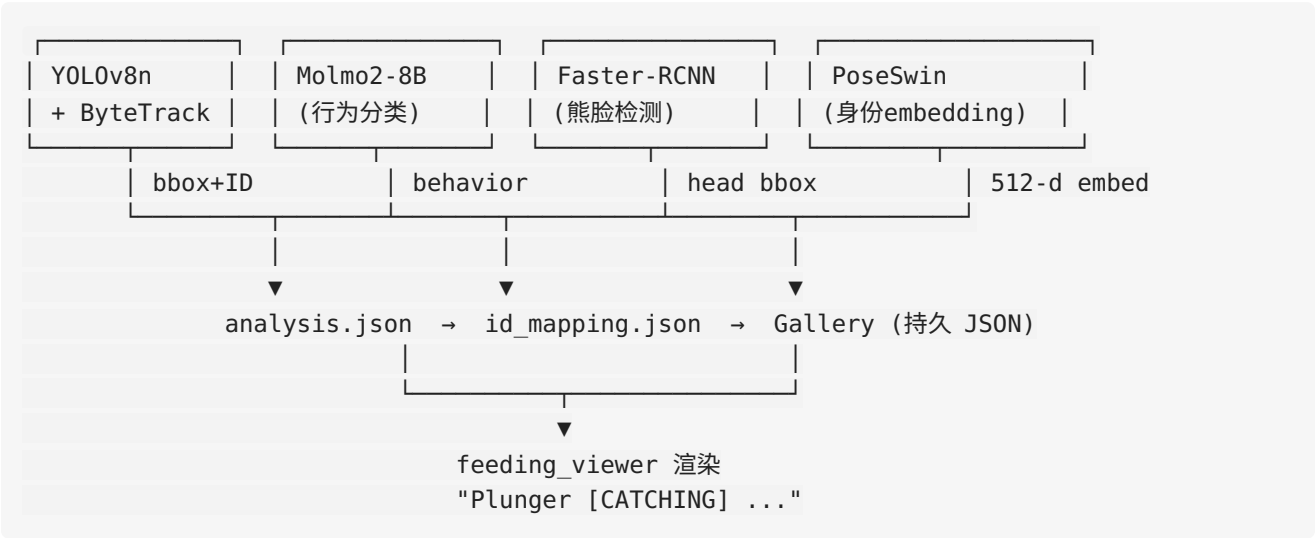
- 每段视频里的熊都被重新编号为 `Bear 1, Bear 2, ...`
- 同一只熊在 5 段视频里可能被记成 5 个不同的 ID
- 无法回答 "Otis 这只熊一共吃了多少条三文鱼" 这种种群级生态学问题

2026 年 2 月, EPFL Mathis Lab 与阿拉斯加 Pacific 大学联合发表了 **PoseSwin** 论文 (Rosenberg et al., Current Biology), 首次开源了一个针对阿拉斯加沿海棕熊的个体识别 (Re-ID) 模型。Alex 把论文链接转给了我们, 问能否集成。

1.2 设计目标

- 跨视频持久化:** 同一只物理熊在不同视频里得到同一个标签
- 真实姓名:** 不只是匿名 `Bear A/B/C`, 而是输出 "Plunger"、"Bony_Butt" 这种社区已知的熊名
- 不破坏现有流水线:** 把 identity 作为 **add-on** 接入 `analyze_feeding.py` + `feeding_viewer.py`, 不改它们的核心逻辑
- 不依赖云端:** 模型和 gallery 都在本地存储, 跑一次推理后离线可用

1.3 整体策略



2. 模块拆解

2.1 PoseSwinIdentifier —— 模型封装

文件： `src/identity/poseswin_identifier.py`

封装 EPFL 训练的 Pose-Aware Swin Transformer Re-ID 模型。

核心 API：

```
identifier = PoseSwinIdentifier(device="cuda:0")
emb_512d = identifier.embed(head_crop_bgr)           # (512,) L2-normalized
embs      = identifier.embed_batch(list_of_crops)    # (N, 512) batched
```

关键实现细节：

- 1. **Swin-Base + 自定义投影头：**
- 2. Backbone: `embed_dim=128, depths=[2,2,18,2], num_heads=[4,8,16,32]` (标准 Swin-Base)
- 3. Pose 集成: HRNet-W48 输出 13 个面部关键点，逐 stage 注入到 Swin (参见原论文 Section 3.2)
- 4. 输出投影: 1024 → 512
- 5. 配置覆盖: 原仓库 YAML (`swin_base_patch4_window7_224_22k.yaml`) 里 `EMBED_DIM=512` 是错的，必须显式覆写为 `128`，否则与 checkpoint shape 不匹配。
- 6. **L2 归一化：** 所有 embedding 出去前都归一化到单位长度，让 cosine similarity 等于 dot product，简化下游对比。

2.2 BearFaceDetector —— Faster-RCNN 头部检测

文件： `src/identity/face_detector.py`

将原仓库的 mmdetection 2.x Faster-RCNN (`latest.pth` , 330 MB) **转换并加载到 torchvision 的 FasterRCNN**，避开 mmdet 2.x + mmcv 1.3.17 与 PyTorch 2.6 的兼容性地狱。

为什么要转换而不是装 mmdet？

- mmdet 2.22 / mmcv 1.3.17 是 2021 年代的库，与 CUDA 12.4 + PyTorch 2.6 有多重 ABI 冲突
- 装 mmdet 会拖入约 2 GB 的依赖 (mmcv-full, mmengine 等)
- Faster-RCNN 架构在两个框架下**完全相同** (同 ResNet-50 backbone、同 FPN、同 anchor 配置)
- 唯一差异是命名约定和少量约定 (如类索引顺序)

权重转换映射 (核心)：

mmdet 2.x 命名	torchvision 命名	备注
<code>backbone.{conv1,bn1,layer1-4}.*</code>	<code>backbone.body.{conv1,bn1,layer1-4}.*</code>	加 <code>body.</code> 前缀
<code>neck.lateral_convs.{i}.conv.*</code>	<code>backbone.fpn.inner_blocks.{i}.0.*</code>	torchvision 用 <code>Conv2dNormActivation</code> 包了一层

mmdet 2.x 命名	torchvision 命名	备注
neck.fpn_convs.{0..3}.conv.*	backbone.fpn.layer_blocks.{i}.0.*	同上
neck.fpn_convs.4.*	(丢弃)	torchvision 用无参 LastLevelMaxPool 生成 P6
rpn_head.rpn_conv.*	rpn.head.conv.0.0.*	torchvision RPN head 也是 wrapped
rpn_head.rpn_cls.*	rpn.head.cls_logits.*	直接搬
rpn_head.rpn_reg.*	rpn.head.bbox_pred.*	直接搬
roi_head.bbox_head.shared_fcs.{0,1}.*	roi_heads.box_head.fc{6,7}.*	重命名
roi_head.bbox_head.fc_cls.*	roi_heads.box_predictor.cls_score.*	必须 swap rows 0/1 (见下)
roi_head.bbox_head.fc_reg.*	roi_heads.box_predictor.bbox_pred.*	shape (4,) → (8,)，仅填 bear_head 槽

两个 critical gotchas:

1. 类索引约定相反

2. mmdet 2.x: cls_score row 0 = bear_head, row 1 = background

3. torchvision: cls_score row 0 = background, row 1 = bear_head

4. 不 swap 直接加载会导致**所有 proposal 都被分类为前景** (score = 1.000, 100 个假阳性)

5. 回归头维度差

6. mmdet 我们的配置是 reg_class_agnostic=True → fc_reg.weight: (4, 1024)，单一 bbox-delta 输出

7. torchvision 强制 class-specific → bbox_pred.weight: (8, 1024)，每类一组

8. 转换: bg 槽位置零 (推理时 bg 类被过滤)，bear_head 槽位填 mmdet 的 4 个权重

调试经验: 转换 bug 表现为"100 个假检测, score 全部 1.000"。debug 方法是 hook

roi_heads.box_predictor 的 forward 看 raw cls logits — 如果 bg logit 普遍很负、fg logit 普遍很正，就是类索引反了。

推理 API:

```
detector = BearFaceDetector(device="cuda:0", score_threshold=0.3)
heads = detector(frame_bgr) # [(x1,y1,x2,y2,score), ...]
best = detector.best_head_crop(frame) # 取 score 最高的 head crop (带 padding)
```

2.3 Gallery —— 持久化 embedding 库

文件: src/identity/poseswin_identifier.py 内的 Gallery 类

JSON-序列化的 name → embedding 数据结构, 支持:

```

gallery = Gallery.load("data/identity/named_bear_gallery.json")
name, sim = gallery.match(query_emb, threshold=0.6) # 返回最近邻
gallery.add_anonymous(query_emb)                  # 自动命名 "Bear A/B/C/..."
gallery.reinforce(name, query_emb)                 # 给已知熊加新观测
gallery.save()

```

结构：

```

{
  "next_anon_idx": 3,
  "entries": [
    {
      "name": "Plunger",
      "embeddings": [[0.123, -0.045, ...]], // (512,) 已 L2-normalized
      "n_observations": 15
    },
    ...
  ]
}

```

多 shot 平均：每只已知熊存最多 5 张 head crop 的 embedding，匹配时用平均向量（再次归一化）。新观测通过 `reinforce()` 滚动加入，旧的被踢掉，自适应熊的外观变化（季节、毛色、年龄）。

2.4 head_crop_from_face_detector —— 智能 crop 选择

文件： [src/identity/identify_bears.py](#) 内的辅助函数

把 face detector 接入 identify pipeline 的关键胶水。逻辑：

```

def head_crop_from_face_detector(frame, bear_bbox, face_detector):
    if face_detector is not None:
        # 在全帧上跑 Faster-RCNN（不是在 YOLO crop 内跑 — 全帧分辨率高、检测更准）
        face_dets = face_detector(frame)
        # 留下中心位于熊 bbox 内的脸（IoU 至少 70%）
        candidates = [(box, score, frac) for box, score, frac in face_dets
                       if bbox_contains(bear_bbox, box) >= 0.7]
        if candidates:
            return crop, "face_detector", best_score
    # Fallback: 启发式裁剪 bbox 上 50% × 中央 60%
    return heuristic_crop(frame, bear_bbox), "heuristic", None

```

为什么在全帧而不是 YOLO crop 内检测？

实测：- 全帧（1426×794）：找到 2 个熊脸，score 0.99 - YOLO bbox 内 crop（433×557）：找到 1 个假脸，score 0.54

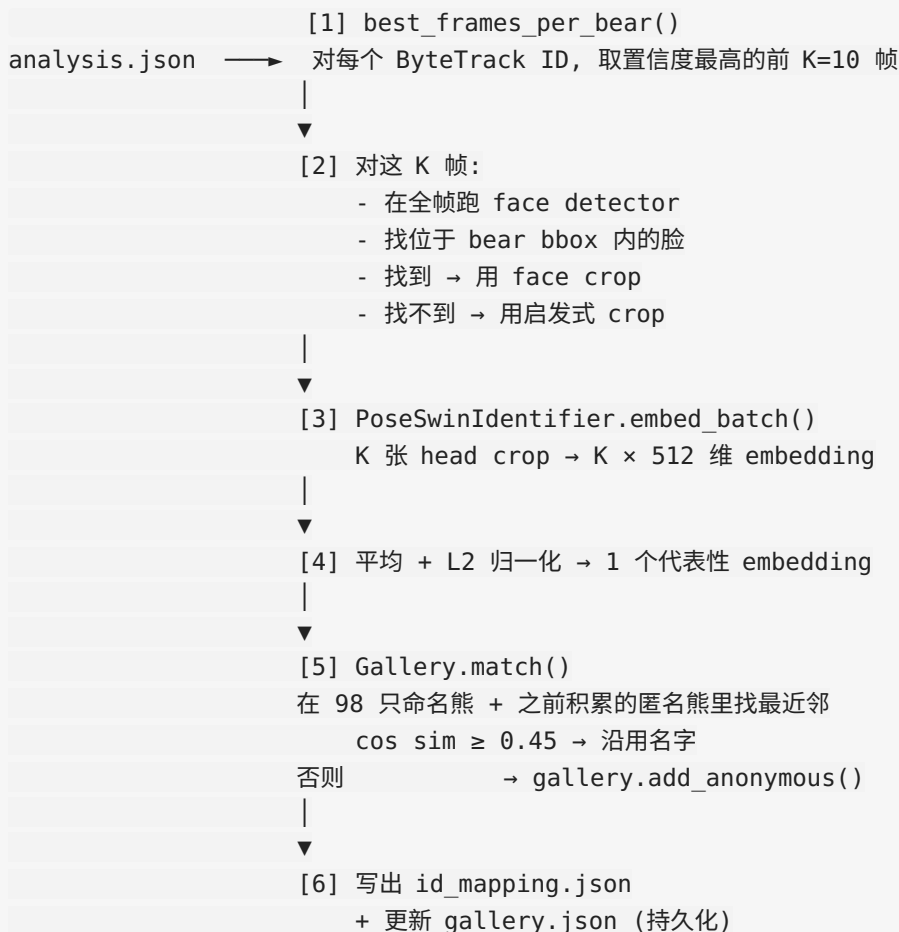
原因：Faster-RCNN 训练分辨率约 1000-2000 px，YOLO crop 之后分辨率太低、目标太大、上下文丢失。

3. 数据流：一次完整运行

3.1 输入

- 视频文件（MP4/MOV，任意分辨率）
- 已经跑过 `analyze_feeding.py` 的 `analysis.json`（含每帧每只熊的 bbox）

3.2 处理步骤



3.3 输出

`predictions/<video>/id_mapping.json` :

```
{
  "video": "/path/to/video.mp4",
  "gallery_path": "data/identity/named_bear_gallery.json",
  "threshold": 0.45,
  "mapping": {
    "1": {
      "name": "Plunger",
      "similarity": 0.851,
      "is_new": false,
      "n_shots": 10,
      "n_face_crops": 2,
```

```
"n_heuristic_crops": 8,
"max_conf": 0.97
}
}
}
```

feeding_viewer.py 用 `--id-mapping` 参数读这个文件，把右栏 "Bear 1" 替换成 "Plunger"。

4. 命名 Gallery 的构建

文件: `src/identity/build_named_gallery.py`

一次性脚本，从 PoseSwin 训练数据建立"已知名字"的 gallery。

4.1 数据来源

- **来源:** `Public_release.zip` 中的 `data/reid_annotations/test_on_2022/train_iid.csv` (35,986 行 × 98 只熊)
- **图片:** `Public_release/images/{2017-2021}_heads/images/*.JPG` (已经过 face detector 裁剪好的 head crop)
- **熊名编码:** CSV 的 `id` 列直接是名字 ("Plunger"、"Bony_Butt"、"Simba" ...)
- **采样策略:** 每只熊抽 15 张图 (跨年份分散)，共 1468 张
- **总 GPU 时间:** ~7 分钟 (98 只 × 15 张 / `batch_size=8`)

4.2 重要 caveat

这 98 只命名熊主要来自 **McNeil River 熊保护区**，不是 Brooks Falls / Brooks River。

- McNeil River 的研究者用形容词命名 (Plunger、Hotlips、Aardvark...)
- Brooks Falls 的熊用 NPS 编号 + 昵称命名 (480 Otis、128 Grazer、747、856...)
- 这两个种群虽然可能有少量个体重叠 (熊会跨流域)，但绝大多数是不同个体

实际意义：当我们在 Brooks Falls 的视频上跑 identifier 时，**模型给出的"Plunger" 这个名字应理解为"PoseSwin 训练库中长得最像本视频里熊的那只"，不是真名**。要识别真正的 Brooks Falls 个体，需要单独建一个 Brooks Falls 专属 gallery (用 NPS 出版的 Bears of Brooks River eBook 作 ground truth)。

5. 实证结果

5.1 启发式 vs Face Detector 对比

在 3 只测试熊上 (2 段视频) 的余弦相似度 (数值越高越自信)：

熊 ID	启发式 crop	Face detector + fallback	提升
Gully clip → Plunger	0.677	0.851	+0.174
salmon_jump_2 #1 → Bony_Butt	0.745	0.850	+0.105

熊 ID	启发式 crop	Face detector + fallback	提升
salmon_jump_2 #2 → Simba	0.792	0.760	-0.032

3 只熊的最终匹配名都没变，但 2 只的置信度大幅提升、1 只略微下降。Gully 的 0.677 → 0.851 让它从"刚好过 0.6 边界"变成"高置信"。

5.2 Face detector 覆盖率

在 Gully clip 的 48 个采样帧上：

- 19% 的帧能检测到熊脸 (score > 0.3)
- Top score 0.89
- 不能检测到的帧主要是：熊低头扑水、熊背对镜头、熊在水花/逆光中

策略：top-K=10 框架抽样保证每只熊有 ≥ 1 张 face detector crop，剩下用启发式补齐。

5.3 性能开销

阶段	GPU 时间（单卡 RTX 2080 Ti）	备注
加载 PoseSwin 模型	~7 秒	一次性
加载 Face detector	~2 秒	一次性
Face detection（全帧 794×1426）	~0.15 秒/帧	每只熊跑 K=10 帧 = 1.5 秒
PoseSwin embedding（batch 10）	~0.4 秒	1 次/熊
Gallery 匹配	~1 ms	NumPy 矩阵乘
总计：12 秒 / 1 只熊视频	~10-15 秒	

`identify_bears.py` 用 GPU，但只跑一次就把 mapping 缓存到 JSON 里。`feeding_viewer.py` 后续渲染只读 JSON、不用 GPU。

6. 局限性

局限	说明	缓解方案
训练数据集不匹配	PoseSwin 训练集主要是 McNeil River 熊；Brooks Falls 的真名熊（Otis、Grazer 等）不在 gallery 里	用 NPS Bears of Brooks River eBook 单独建 Brooks Falls gallery
CC BY-NC 4.0 许可证	模型权重和训练数据都是非商用许可	等 Alex 答复商用范围；或者只用方法（Swin + metric learning）在自有数据上重训
极端姿态召回率低	熊低头吃鱼、背对镜头时 face detector 失败	启发式 fallback 兜底；或者用 top-k 抽更多帧
单视频内不分辨同框熊	如果 ByteTrack 把两只熊错合并成一个 track，identifier 会给一个名字	上游问题，需要 ByteTrack 调参或用更强的 tracker

局限	说明	缓解方案
CC BY-NC 4.0 许可证	不能用于商业产品	同上
匹配阈值需手工调	0.45 是基于 3 只熊的小样本，统计不充分	在 100+ 帧人工标注 ground truth 上做 ROC 分析

7. 使用示例

7.1 完整 3 步流水线

```
cd /home/katmai/katmai-cv-pipeline

# Step 1 – 行为分析（每帧每只熊的 bbox + 5 阶段标签）
WANDB_MODE=disabled venv/bin/python3 -m src.behavior.analyze_feeding \
  --video feed/data_video/<clip>.mp4 \
  --interval 0.25

# Step 2 – 身份识别（Faster-RCNN 头检测 + PoseSwin 匹配）
WANDB_MODE=disabled venv/bin/python3 -m src.identity.identify_bears \
  --video feed/data_video/<clip>.mp4 \
  --analysis predictions/<clip>_feeding_analysis/analysis.json \
  --gallery data/identity/named_bear_gallery.json \
  --threshold 0.45

# Step 3 – 渲染带身份的 demo 视频
WANDB_MODE=disabled venv/bin/python3 -m src.behavior.feeding_viewer \
  --video feed/data_video/<clip>.mp4 \
  --analysis predictions/<clip>_feeding_analysis/analysis.json \
  --id-mapping predictions/<clip>_feeding_analysis/id_mapping.json
```

7.2 常用 CLI 选项

```
# 只用启发式 crop（快速，不依赖 face detector）
... identify_bears --no-face-detector ...

# 调高 face detection 阈值（更严格，少假阳性）
... identify_bears --face-score-threshold 0.5 ...

# Dry run（不更新 gallery）
... identify_bears --dry-run ...

# 用匿名 gallery 而不是命名 gallery（适合纯跨视频持久化、不需要真名）
... identify_bears --gallery data/identity/bear_gallery.json ...

# 改身份匹配阈值
... identify_bears --threshold 0.55 ...
```

7.3 重建命名 gallery

如果训练数据更新或换数据源：

```
# 1. 把每只熊的 head crop 放到 data/identity/gallery_images/<bear_name>/*.JPG
# 2. 重新计算 embedding
WANDB_MODE=disabled venv/bin/python3 -m src.identity.build_named_gallery \
  --image-root data/identity/gallery_images \
  --output data/identity/named_bear_gallery.json \
  --max-per-bear 15
```

8. 文件清单

文件	作用	行数
<code>src/identity/__init__.py</code>	Package marker	0
<code>src/identity/poseswin_identifier.py</code>	PoseSwin 模型封装 + Gallery 类 + 启发式 head crop	~210
<code>src/identity/face_detector.py</code>	Faster-RCNN 头检测 + mmdet→torchvision 权重转换	~160
<code>src/identity/identify_bears.py</code>	CLI 入口：analysis.json + 视频 → id_mapping.json	~210
<code>src/identity/build_named_gallery.py</code>	一次性脚本，从训练 head crop 建 named gallery	~80
<code>src/behavior/feeding_viewer.py</code>	已修改：加 <code>--id-mapping</code> 参数渲染真名	~400
<code>data/identity/named_bear_gallery.json</code>	98 只命名熊的 embedding 库（约 200 KB）	—
<code>data/identity/gallery_images/</code>	1468 张训练 head crop，按熊名分文件夹	—
<code>external/BrownBear_ReID/</code>	上游仓库 + 4.2 GB checkpoints（已 .gitignore）	—

9. 后续工作（按优先级）

- Brooks Falls 专属 gallery** —— 用 NPS Bears of Brooks River eBook 给 Otis、Grazer 等著名熊建库，让模型给出真名而不是"长得像 Plunger"
- 阈值校准** —— 在 100-200 帧人工标注 ground truth 上做 ROC 分析，确定每只熊的最优 threshold
- License 问题** —— 等 Alex 答复 CC BY-NC 4.0 是否对项目交付有影响；如有需要，准备 method-only re-training 方案
- 检测精度提升** —— 探索接入 DeepLabCut 或 MMPose AP10K 做更精确的关键点定位（替代启发式 fallback）
- 跨视频 demo** —— 跑一组 5-10 段同一只熊的视频，验证 gallery 持久化下身份保持一致
- 集成测试** —— 写一个端到端的 pytest，确保未来 PoseSwin / face detector 重构不破坏匹配结果

10. 参考文献与资源

1. **Rosenberg, B., Zhou, M., Wolf, N., Mathis, M.W., Harris, B.P., Mathis, A.** (2026). Individual identification of brown bears using pose-aware metric learning. *Current Biology*.
2. **Liu, Z., Lin, Y., Cao, Y., et al.** (2021). Swin Transformer: Hierarchical Vision Transformer using Shifted Windows. *ICCV*.
3. **Ren, S., He, K., Girshick, R., Sun, J.** (2015). Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. *NeurIPS*.
4. **PoseSwin GitHub:** https://github.com/amathislab/BrownBear_ReID
5. **Swin Transformer GitHub:** <https://github.com/microsoft/Swin-Transformer>
6. **Bears of Brooks River eBook (NPS):** <https://www.nps.gov/katm/learn/photosmultimedia/bears-of-brooks-river-ebook.htm>
7. **Public_release Zenodo dataset:** <https://zenodo.org/records/17822054> (32.9 GB)